

Proyuts Tracker

BRAYAN JULIAN BENAVIDES CASTRO

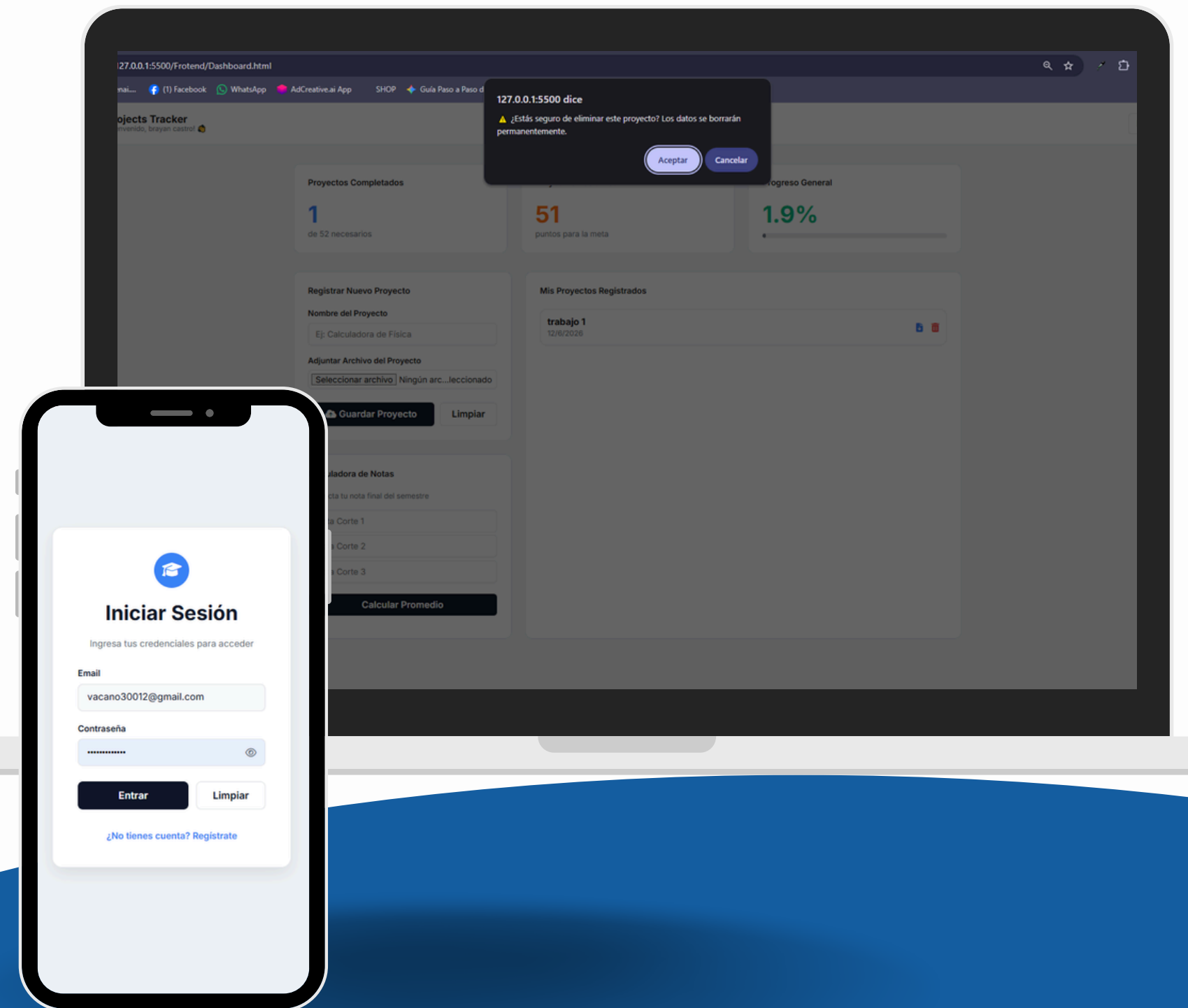


Contenido

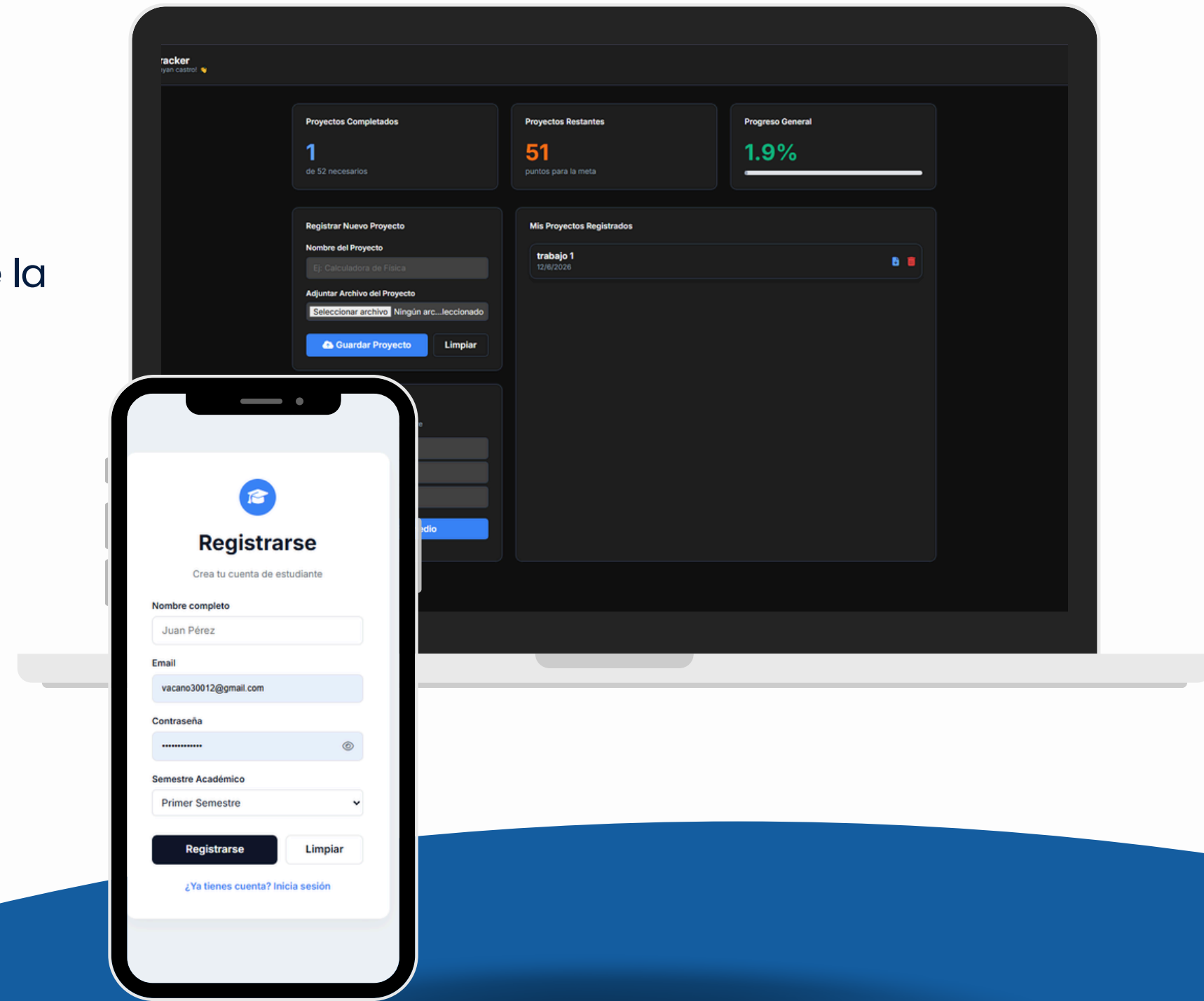
▶▶	Qué hice – 10 funcionalidades – JIRA	01
▶▶	Cómo lo hice – Código – GitHub	02
▶▶	Dificultades – APP	03
▶▶	Cómo se aplicó la IA	04
▶▶	Trabajos futuros	05

Funcionalidades

- Calculadora automática de proyectos faltantes en la pantalla principal para saber exactamente cuántos puntos restan para llegar a la meta.
- Alerta de confirmación emergente antes de eliminar un proyecto para evitar que el estudiante borre su información por accidente.
- Validación de campos vacíos en los formularios de registro para que el sistema bloquee el guardado si al estudiante se le olvida llenar un dato.
- Botón de limpiar formulario para borrar rápidamente todo lo escrito en las casillas con un solo clic.
- Restricción de seguridad en el formulario de registro que obligue a que la contraseña creada tenga un mínimo de caracteres.



- Función de mostrar y ocultar contraseña en la pantalla de inicio de sesión para verificar lo que se escribió antes de entrar.
- Sistema de Modo Oscuro / Modo Claro (Dark Theme), con un botón interruptor que cambie la paleta de colores de toda la interfaz y guarde la preferencia en la memoria del navegador
- Mensaje de bienvenida personalizado en la parte superior del menú principal que muestre el nombre del estudiante una vez ingrese al sistema.
- Bloqueo automático del botón de inscripción a cursos si el sistema detecta que el estudiante ya alcanzó o superó sus 52 proyectos requeridos.
- Calculadora de Notas interactiva, donde el estudiante pueda ingresar sus calificaciones y los porcentajes de cada corte para proyectar su nota final del semestre.



Arquitectura y Capas del Sistema

01

Interfaz (Frontend): Construcción visual de la plataforma utilizando HTML5 para la estructura de las vistas y CSS3 para el diseño estético.

02

Interactividad (JavaScript): Uso de JavaScript nativo para dar vida a la página, permitiendo capturar de forma dinámica los datos que el usuario digita en los formularios de inicio de sesión y registro.

03

Servidor (Backend - Node.js): Creación del motor del sistema sobre Node.js y el framework Express.js, encargado de recibir la información enviada por la interfaz y decidir qué acciones debe ejecutar el programa.

04

Control y Validación: Implementación de filtros básicos en el servidor que verifican que el usuario no envíe campos vacíos, procesando las solicitudes de manera organizada antes de tocar la base de datos.

Almacenamiento e Infraestructura Local

05

Base de Datos (PostgreSQL): Uso de un sistema organizado en la computadora para almacenar de forma permanente toda la información del proyecto, como las cuentas de los usuarios y los registros de los estudiantes.

06

Conexión del Sistema: Configuración de un puente directo entre el servidor y la base de datos, permitiendo que la información se guarde o se consulte de inmediato cada vez que el usuario interactúa con la plataforma.

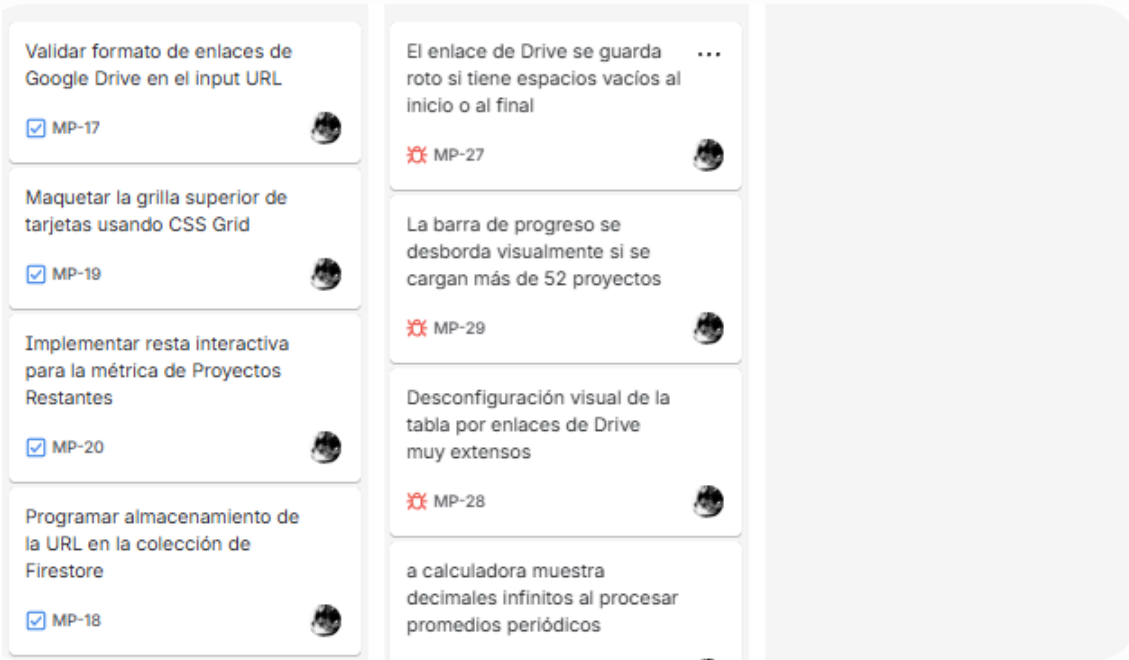
07

Uso de Docker para empaquetar el servidor y la base de datos dentro de "cajas" virtuales en la computadora, logrando que todos los programas necesarios corran juntos sin conflictos.

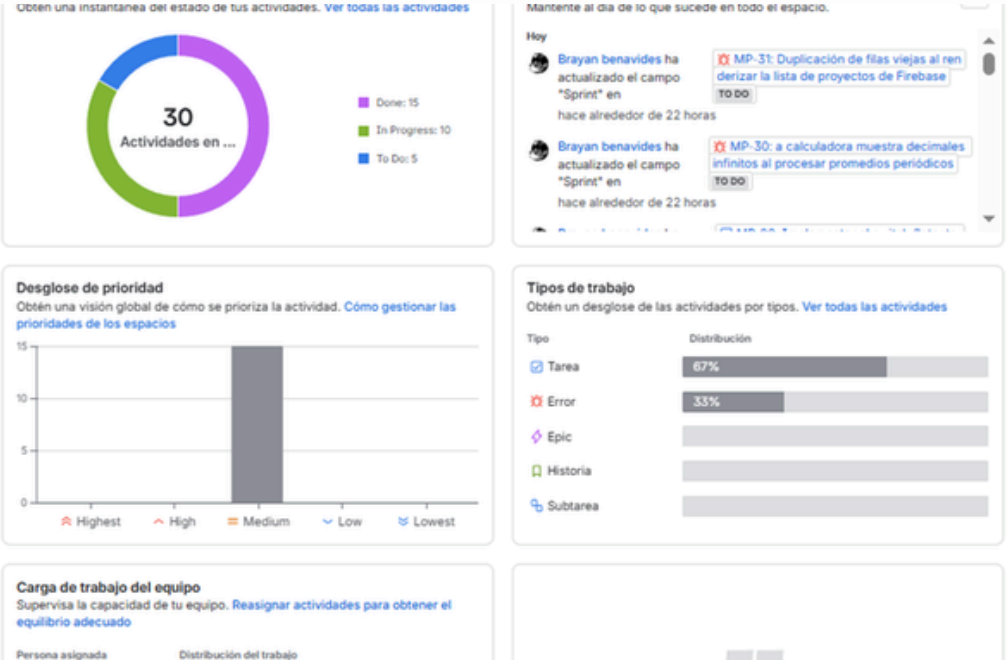
08

Uso de un archivo de configuración oculto para guardar de forma segura las contraseñas internas y las rutas de la máquina, protegiendo el acceso al sistema desde el propio equipo.

JIRA



Organización visual del flujo diario en localhost mediante columnas de estado. Permite gestionar en tiempo real las actividades distribuidas a lo largo de los dos Sprints de trabajo, facilitando el desarrollo de los módulos activos y el aislamiento estratégico de los bugs detectados en cada ciclo.



Panel de control para el monitoreo gráfico y medición de la salud del proyecto. Muestra el balance y rendimiento de las 30 actividades ejecutadas durante los dos Sprints, detallando que el 67% del esfuerzo se dedicó a nuevas funcionalidades y el 33% restante al control de errores de cada ciclo.

A screenshot of the Jira board view. It shows a Kanban board with tasks in different stages: To Do, In Progress, and Done. The tasks are: 'Calculadora de proyectos faltantes', 'Alerta de confirmación al eliminar', 'Validación de campos vacíos', 'Botón de limpiar formulario', 'Mostrar y ocultar contraseña', 'Sistema de Modo Oscuro / Claro', 'Mensaje de bienvenida personalizado', 'Bloqueo de inscripción por límite de proyectos', 'Restricción de seguridad en contraseña', 'Calculadora de Notas Interactiva', 'Vista del Historial de Proyectos Registrados', 'Registro de Nuevos Estudiantes en la Plataforma', 'Cierre de Sesión Seguro', 'Persistencia de datos del estudiante', 'Implementar el almacenamiento del enlace de ...', 'Validar formato de enlaces de Google Drive en ...', 'Maquetar la grilla superior de tarjetas usando ...', 'El enlace de Drive se guarda roto si tiene espa...', 'Implementar resta interactiva para la métrica ...', 'La barra de progreso se desborda visualmente ...', 'Programar almacenamiento de la URL en la col...', 'Desconfiguración visual de la tabla por entice...', 'Configurar interceptor en el formulario para evi...', and 'Configurar marcadores de posición (Placeholder...)'. Each task has a checkbox, a progress indicator, and a status label.

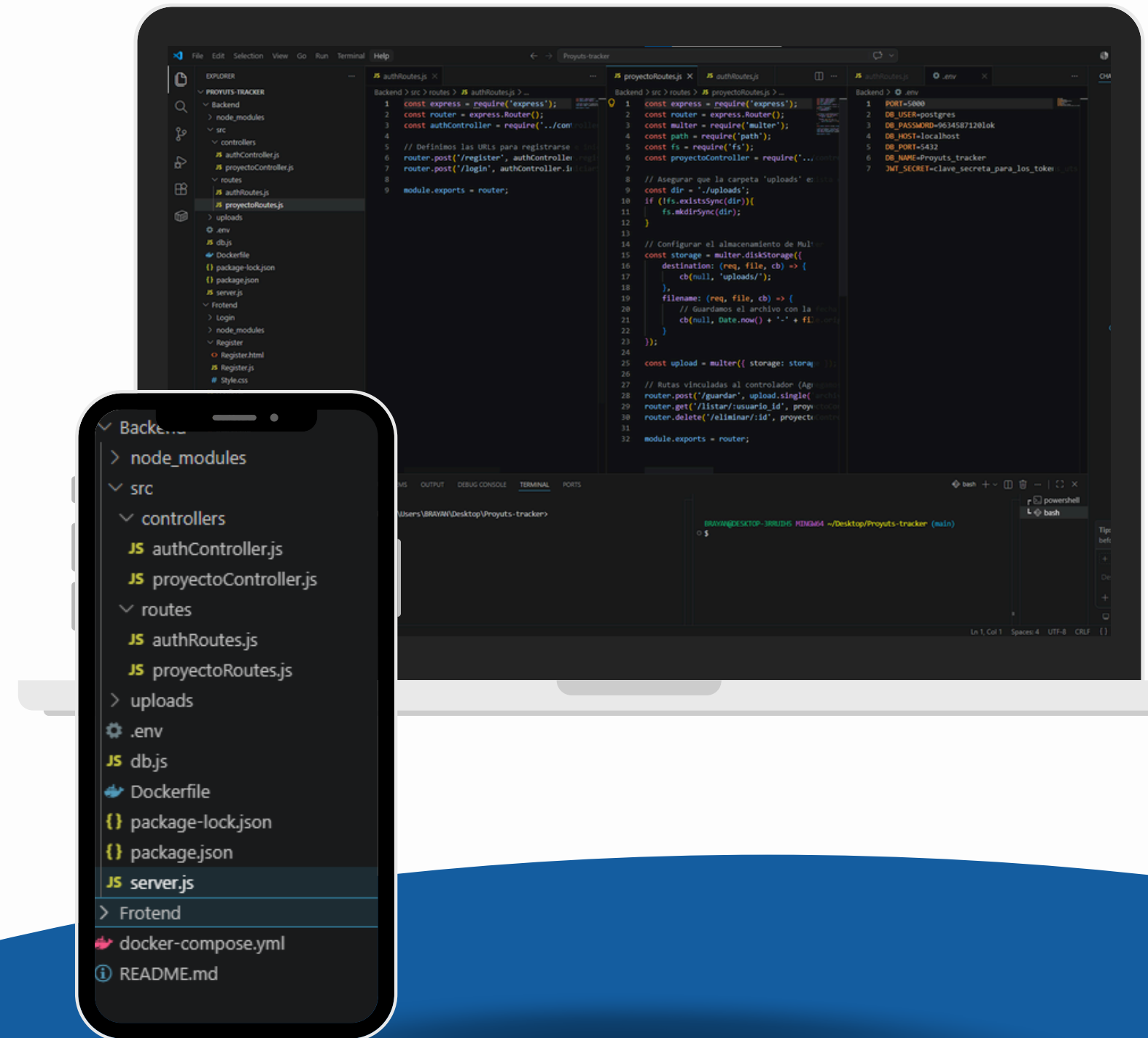
Vista de planificación centralizada que recopila las 30 tareas clave mapeadas en el cronograma. Evidencia la culminación exitosa de los requerimientos base y la transición ordenada hacia los dos Sprints de desarrollo activos, donde se procesaron las optimizaciones finales del sistema.

Arquitectura Limpia y Stack Tecnológico

Construcción del sistema dividiendo el código en dos repositorios independientes. El Frontend se programó utilizando tecnologías web nativas como HTML5, CSS3 y JavaScript para garantizar una interfaz de usuario fluida y reactiva. Por otro lado, el Backend gestiona la lógica del servidor y el almacenamiento relacional de los datos mediante una base de datos en PostgreSQL, todo estructurado y contenedorizado de forma eficiente con Docker para asegurar un entorno de desarrollo local limpio y estandarizado.

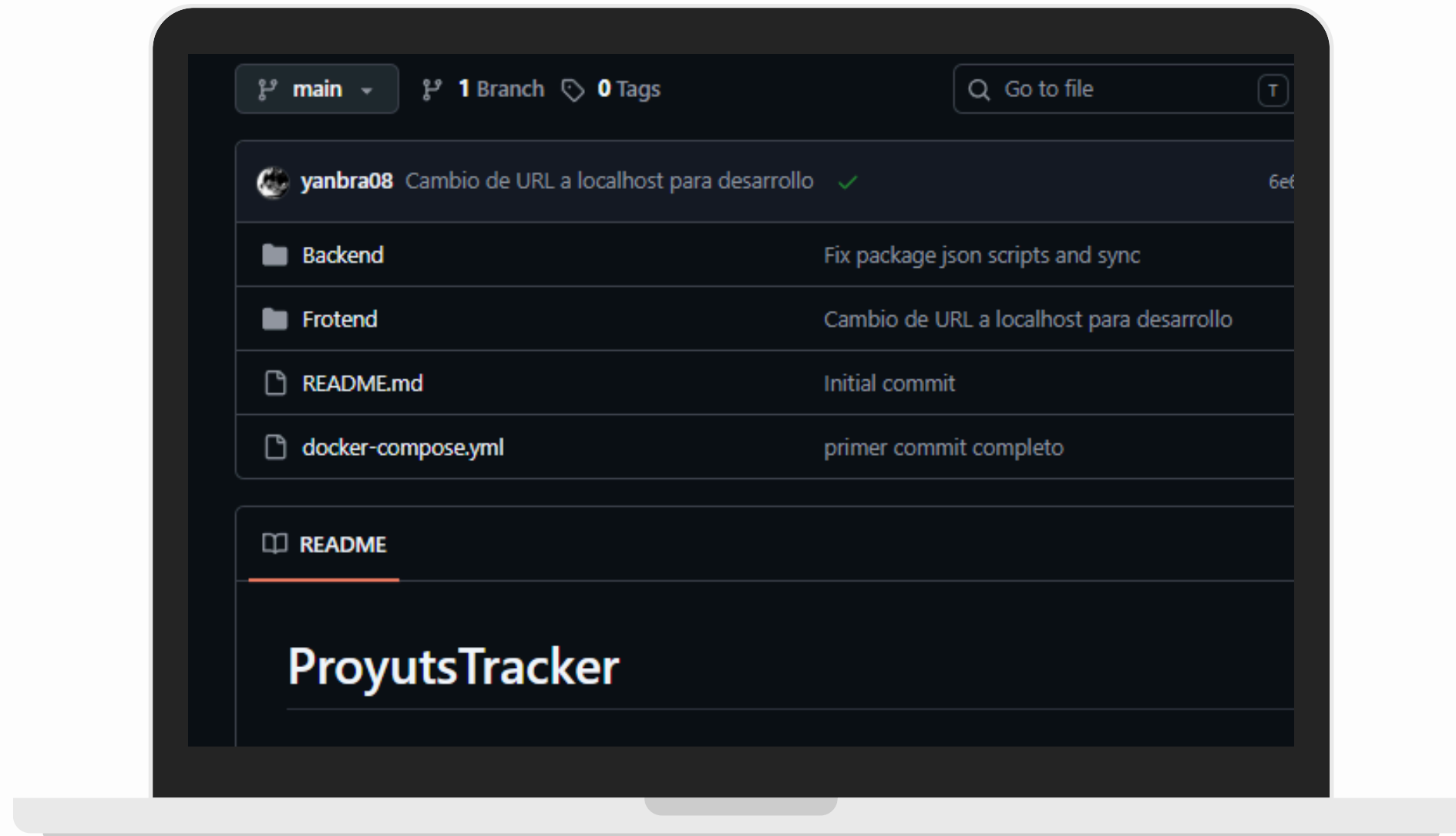


- Arquitectura Limpia (MVC): Estructuración modular en el Backend separando las rutas (routes) y la lógica de negocio en controladores (controllers) como authController.js y proyectoController.js.
- Seguridad y Configuración: Implementación de variables de entorno mediante el archivo .env para proteger las credenciales de conexión de la base de datos mapeada en db.js.
- Contenedores de Docker: Uso de un Dockerfile específico para empaquetar el servidor de Node.js (server.js) y un docker-compose.yml en la raíz para orquestar de manera conjunta el Backend, el Frontend y PostgreSQL.



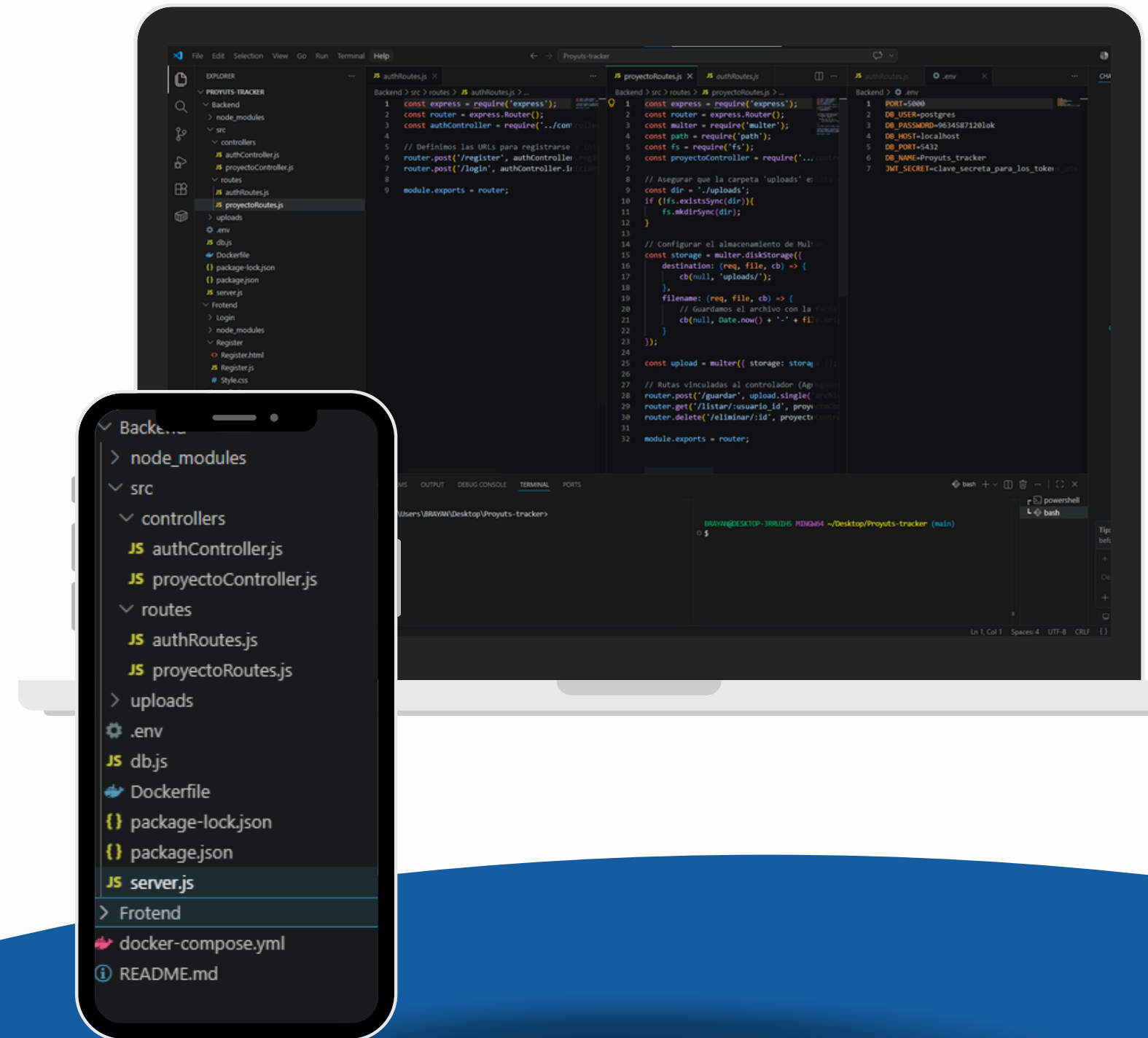
GITHUB

- Todo el desarrollo diario realizado en localhost se gestionó mediante Git para registrar de forma ordenada el avance de las 30 tareas estructuradas en los dos Sprints de JIRA. Los cambios y optimizaciones se agruparon por bloques de avance funcional mediante confirmaciones (commits) estratégicas, sincronizando de forma periódica el código con la nube en el repositorio seguro PROYUTS-TRACKER de GitHub para garantizar la disponibilidad y el control de cambios del proyecto.



IA

- Optimización y Depuración de Código: Uso de la IA como herramienta de soporte para la refactorización de funciones repetitivas y la detección rápida de errores de sintaxis en localhost.
- Soporte en Configuración de Entornos: Asistencia en la estructuración base de archivos técnicos complejos como el Dockerfile y el docker-compose.yml.
- Identidad Visual del Proyecto: Aplicación de modelos generativos de IA para la conceptualización, diseño y creación del logo oficial de la plataforma.



Dificultades

- Problema de Despliegue Exterior: El mayor reto técnico del proyecto fue configurar y estabilizar la comunicación directa entre el Backend (alojado en Render) y el Frontend (alojado en Netlify).
- Configuración del Entorno Virtual (Docker): Dificultad inicial al mapear correctamente los volúmenes en el Dockerfile para que el servidor de Node.js reconociera los cambios de código en tiempo real dentro del contenedor.
- Conexión de la Base de Datos Relacional: Errores de autenticación y de red local al enlazar el archivo db.js del Backend con el puerto expuesto de PostgreSQL corriendo en el entorno local.
- Control de Asincronismo en las Peticiones: Manejo y depuración de promesas inconclusas en JavaScript al realizar peticiones de inicio de sesión (authController.js) y gestión de proyectos, lo que generaba respuestas vacías en la interfaz.

Sugerencias y Recomendaciones

- Migración Arquitectónica: Evaluar a mediano plazo la implementación de frameworks robustos en el Frontend (como React o Angular) para facilitar la escalabilidad del sistema.
- Transición a Entornos Cloud: Recomendar la migración del ecosistema de desarrollo local hacia servidores de hosting remoto en la nube (como Render, Netlify o AWS) para habilitar el acceso público a la plataforma mediante un dominio web.
- Estandarización con Docker: Mantener el uso estricto de contenedores Docker en todas las fases del ciclo de vida del software para asegurar que el despliegue final en producción sea idéntico al entorno local actual.

Trabajos Futuros

- **Despliegue y Configuración de Infraestructura:** Ejecutar el despliegue definitivo en servidores web e implementar políticas avanzadas de origen cruzado (CORS) para asegurar la comunicación fluida entre el Frontend y el Backend en producción.
- **Módulo de Analítica Avanzada:** Desarrollar un panel de control con reportes estadísticos y gráficos interactivos para el seguimiento del rendimiento de los estudiantes en sus proyectos universitarios.
- **Sistema de Notificaciones Automatizadas:** Integrar un servicio de alertas por correo electrónico o mensajería instantánea para notificar a los usuarios sobre cambios de estado o entregas pendientes en JIRA o en la plataforma

MUCHAS GRACIAS

